Code Review Report

CSE 3902 Hollow Knight Clone Team Pie

Enemies & Obstacles Integration Branch

Review

1 Sri: Item Control

Field	Details
Author of Code Review	Thomas Sobodosh, Sudhish Gopalakrishnan
Date of Code Review	February 15/20, 2026
Sprint Number	Sprint 2
.cs File Reviewed	`CycleItemNextCommand.cs,KeyboardBindings.cs`
Author of .cs File	Sri
Time to Complete Review	20 minutes

Code Snippet

```
using HollowKnight.Interfaces;
using HollowKnight.Storage;

namespace HollowKnight.Commands
{
    public class CycleItemNextCommand : ICommand
    {
        public CycleItemNextCommand() { }

        public void Execute()
        {
            ItemManager.NextItem();
        }
    }
}
```

```
public static class KeyboardBindings
{
    public static void BindGameplay(KeyboardController keyboard, TheKnight knight, Game1 game)
    {
        // Move Left
        keyboard.RegisterHeldCommand(Keys.A, new PlayerMoveLeftCommand(knight));
        keyboard.RegisterHeldCommand(Keys.Left, new PlayerMoveLeftCommand(knight));

        // Move Right
        keyboard.RegisterHeldCommand(Keys.D, new PlayerMoveRightCommand(knight));
        keyboard.RegisterHeldCommand(Keys.Right, new PlayerMoveRightCommand(knight));

        // Look Up
        keyboard.RegisterHeldCommand(Keys.W, new PlayerMoveUpCommand(knight));
        keyboard.RegisterHeldCommand(Keys.Up, new PlayerMoveUpCommand(knight));

        // Look Down
        keyboard.RegisterHeldCommand(Keys.S, new PlayerMoveDownCommand(knight));
        keyboard.RegisterHeldCommand(Keys.Down, new PlayerMoveDownCommand(knight));

        // Stop Horizontal
        keyboard.RegisterReleasedCommand(Keys.A, new PlayerStopMovingHorizontalCommand(knight));
        keyboard.RegisterReleasedCommand(Keys.Left, new PlayerStopMovingHorizontalCommand(knight));
        keyboard.RegisterReleasedCommand(Keys.D, new PlayerStopMovingHorizontalCommand(knight));
        keyboard.RegisterReleasedCommand(Keys.Right, new PlayerStopMovingHorizontalCommand(knight));

        // Stop Vertical
        keyboard.RegisterReleasedCommand(Keys.W, new PlayerStopMovingVerticalCommand(knight));
        keyboard.RegisterReleasedCommand(Keys.Up, new PlayerStopMovingVerticalCommand(knight));
        keyboard.RegisterReleasedCommand(Keys.S, new PlayerStopMovingVerticalCommand(knight));
        keyboard.RegisterReleasedCommand(Keys.Down, new PlayerStopMovingVerticalCommand(knight));

        // Jump
        keyboard.RegisterPressedCommand(Keys.Space, new PlayerJumpCommand(knight));
```

Readability Assessment

What IS readable:

The screenshot of the first class is short and single-purpose, which makes it very easy to understand at a glance. The constructor clearly shows its two dependencies (player and direction), and the Execute method is a single line that immediately communicates what the command does. Following the ICommand pattern makes the intent obvious to anyone familiar with the project.

The function calls in KeyboardBinding.cs are readable with long descriptive names (eg.PlayerStopMovingHorizontalCommand). Having white space and comments is also a nice touch to easily group together certain keybinds.

What is NOT readable:

The lack of XML summary comments in the first image means a new team member would need to read the implementation to understand what this class does rather than getting it from IntelliSense.

Many if not all the lines in the keybinds snippet are similar, meaning that it can likely be abstracted.

Review

Review Comments

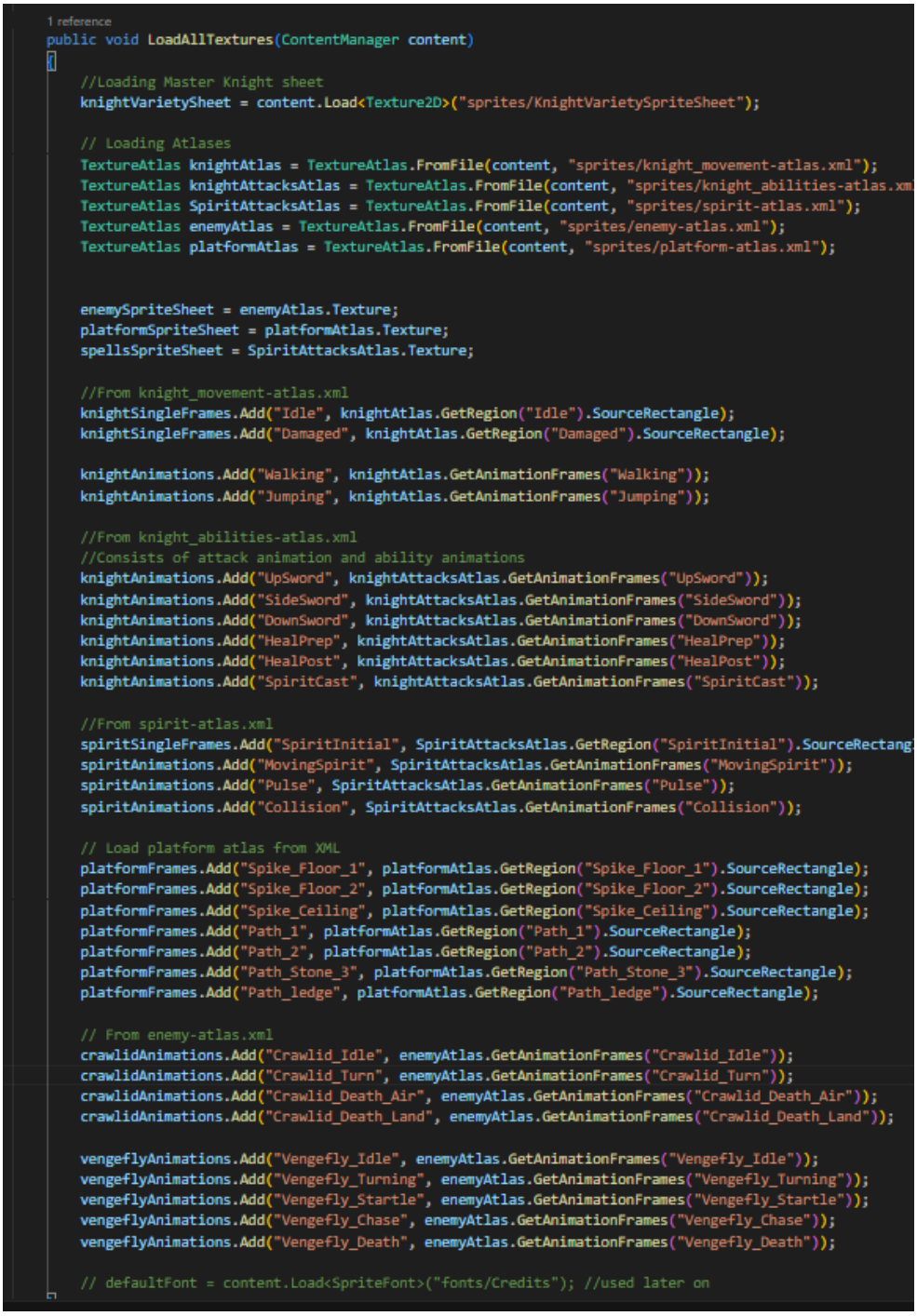
Praise	**File:** `CycleItemNextCommand.cs` Clean implementation of the command pattern. Making the command do a single job is very smart and makes it easier to read and understand. **File:** `KeyBinding.cs` Cleanly written code, easy to follow after a few seconds of looking at the code. Comments and grouping lines also help readability.

Suggestion	**File:** `CycleItemCommand.cs` Add XML summary comments to the class and constructor. All public classes in the project have documentation, and this file is missing it entirely. **Suggestion:** *Add /// <summary> tags describing what the command does* **File:** `KeyBinding.cs` Creating dictionaries for keybinds might be useful to store controls. This will simplify code, since many lines are very similar. Another option would be to have a master function with params: (Key_input, Command(sprite)). **Suggestion:** *Using a master function or dictionary for keybinding and inputs*

2 Thomas & Sudhish: Asset Loading

Field	Details
Author of Code Review	Zach, Sri
Date of Code Review	February 10, 2026
Sprint Number	Sprint 2
.cs File Reviewed	`SpriteFactory.cs`
Author of .cs File	Thomas, Sudhish
Time to Complete Review	20 minutes

Review



```
1 reference
public void LoadAllTextures(ContentManager content)
{
    //Loading Master Knight sheet
    knightVarietySheet = content.Load<Texture2D>("sprites/KnightVarietySpriteSheet");

    // Loading Atlases
    TextureAtlas knightAtlas = TextureAtlas.FromFile(content, "sprites/knight_movement-atlas.xml");
    TextureAtlas knightAttacksAtlas = TextureAtlas.FromFile(content, "sprites/knight_abilities-atlas.xm
    TextureAtlas SpiritAttacksAtlas = TextureAtlas.FromFile(content, "sprites/spirit-atlas.xml");
    TextureAtlas enemyAtlas = TextureAtlas.FromFile(content, "sprites/enemy-atlas.xml");
    TextureAtlas platformAtlas = TextureAtlas.FromFile(content, "sprites/platform-atlas.xml");


    enemySpriteSheet = enemyAtlas.Texture;
    platformSpriteSheet = platformAtlas.Texture;
    spellsSpriteSheet = SpiritAttacksAtlas.Texture;

    //From knight_movement-atlas.xml
    knightSingleFrames.Add("Idle", knightAtlas.GetRegion("Idle").SourceRectangle);
    knightSingleFrames.Add("Damaged", knightAtlas.GetRegion("Damaged").SourceRectangle);

    knightAnimations.Add("Walking", knightAtlas.GetAnimationFrames("Walking"));
    knightAnimations.Add("Jumping", knightAtlas.GetAnimationFrames("Jumping"));

    //From knight_abilities-atlas.xml
    //Consists of attack animation and ability animations
    knightAnimations.Add("UpSword", knightAttacksAtlas.GetAnimationFrames("UpSword"));
    knightAnimations.Add("SideSword", knightAttacksAtlas.GetAnimationFrames("SideSword"));
    knightAnimations.Add("DownSword", knightAttacksAtlas.GetAnimationFrames("DownSword"));
    knightAnimations.Add("HealPrep", knightAttacksAtlas.GetAnimationFrames("HealPrep"));
    knightAnimations.Add("HealPost", knightAttacksAtlas.GetAnimationFrames("HealPost"));
    knightAnimations.Add("SpiritCast", knightAttacksAtlas.GetAnimationFrames("SpiritCast"));

    //From spirit-atlas.xml
    spiritSingleFrames.Add("SpiritInitial", SpiritAttacksAtlas.GetRegion("SpiritInitial").SourceRectang
    spiritAnimations.Add("MovingSpirit", SpiritAttacksAtlas.GetAnimationFrames("MovingSpirit"));
    spiritAnimations.Add("Pulse", SpiritAttacksAtlas.GetAnimationFrames("Pulse"));
    spiritAnimations.Add("Collision", SpiritAttacksAtlas.GetAnimationFrames("Collision"));

    // Load platform atlas from XML
    platformFrames.Add("Spike_Floor_1", platformAtlas.GetRegion("Spike_Floor_1").SourceRectangle);
    platformFrames.Add("Spike_Floor_2", platformAtlas.GetRegion("Spike_Floor_2").SourceRectangle);
    platformFrames.Add("Spike_Ceiling", platformAtlas.GetRegion("Spike_Ceiling").SourceRectangle);
    platformFrames.Add("Path_1", platformAtlas.GetRegion("Path_1").SourceRectangle);
    platformFrames.Add("Path_2", platformAtlas.GetRegion("Path_2").SourceRectangle);
    platformFrames.Add("Path_Stone_3", platformAtlas.GetRegion("Path_Stone_3").SourceRectangle);
    platformFrames.Add("Path_ledge", platformAtlas.GetRegion("Path_ledge").SourceRectangle);

    // From enemy-atlas.xml
    crawlidAnimations.Add("Crawlid_Idle", enemyAtlas.GetAnimationFrames("Crawlid_Idle"));
    crawlidAnimations.Add("Crawlid_Turn", enemyAtlas.GetAnimationFrames("Crawlid_Turn"));
    crawlidAnimations.Add("Crawlid_Death_Air", enemyAtlas.GetAnimationFrames("Crawlid_Death_Air"));
    crawlidAnimations.Add("Crawlid_Death_Land", enemyAtlas.GetAnimationFrames("Crawlid_Death_Land"));

    vengeflyAnimations.Add("Vengefly_Idle", enemyAtlas.GetAnimationFrames("Vengefly_Idle"));
    vengeflyAnimations.Add("Vengefly_Turning", enemyAtlas.GetAnimationFrames("Vengefly_Turning"));
    vengeflyAnimations.Add("Vengefly_Startle", enemyAtlas.GetAnimationFrames("Vengefly_Startle"));
    vengeflyAnimations.Add("Vengefly_Chase", enemyAtlas.GetAnimationFrames("Vengefly_Chase"));
    vengeflyAnimations.Add("Vengefly_Death", enemyAtlas.GetAnimationFrames("Vengefly_Death"));

    // defaultFont = content.Load<SpriteFont>("fonts/Credits"); //used later on
```

Code Snippet

Readability Assessment

What IS readable:

The class structure is immediately clear: private static fields at the top, one load method, then getter methods. The naming is descriptive and consistent. The getter methods follow a predictable Get[Name] pattern, making the API easy to discover. The LoadAllTextures method reads top-to-bottom as a straightforward loading sequence, and the content path strings clearly map to the folder structure.

What is NOT readable:

The one-line getter methods sacrifice readability for brevity. While compact, our project typically uses multi-line method bodies. More importantly, LoadAllTextures has no indication of what happens if a file is missing, a reader would assume the game just crashes, which isn't obvious from reading the code alone. A brief comment or error handling would clarify the expected behavior on failure.

Review Comments

Praise	**File:** `SpriteFacotry.cs` Solid implementation of a factory. The pattern is consistent with the original template, field naming is clear, and it's easy to find any asset by name. Good work keeping the class organized as it grew.
Must Fix	**File:** `SpriteFactory.cs` (LoadAllTextures method) No error handling around content.Load calls. If any sprite sheet is missing or misnamed, the game crashes with an unhelpful ContentLoadException at startup, blocking every team member from testing. This is the single shared load point for the entire project. **Suggestion:** *Wrap each Load call in try-catch, log which asset failed (e.g., System.Diagnostics.Debug.WriteLine), and either throw a clear exception or allow the game to continue with null so other systems can still be tested.*
Should Fix	**File:** `SpriteFactory.cs` The only thing that could be changed is the environment sprite names. They seem unintuitive to someone that is reading the different sprites that can be created. Especially when the other sprites are very detailed with their naming. **Suggestion:** *Change the names of the environment sprites to become more descriptive.*
Suggestion	**File:** `SpriteFactory.cs` As the project grows, consider switching to multiple sprite factories. As the project grows larger, the more sprites that will need to be drawn. Therefore, the size of this file could become massive. A possible fix could be, making multiple

Review

	sprite factories but they could be more specific to what they do. Like an EnemySpriteFactory and a KnightSpriteFactory.

3 Zach: Enemies & Obstacles

Field	Details
Author of Code Review	Nicholas Mamais, Sudhish Gopalakrishnan
Date of Code Review	February 19/20, 2026
Sprint Number	Sprint 2
.cs File Reviewed	`Crawlid.cs`, `CrawlidStateMachine.cs`
Author of .cs File	Zach
Time to Complete Review	35 minutes

Code Snippet

```
using HollowKnight.Factories;

using HollowKnight.Interfaces;

using Microsoft.Xna.Framework;

using Microsoft.Xna.Framework.Graphics;

public class Crawlid : IEnemy

{

    private int frameCounter = 0;

    public int state = 0;

    private CrawlidStateMachine stateMachine;

    public ISprite Sprite;

    public bool left;

    public bool alive;

    public Vector2 position;

    public Crawlid(Vector2 _position)

    {

        position = _position;

        left = true;
```

```
        alive = true;
        Sprite = SpriteFactory.Instance.CreateCrawlidIdleSprite(position);
        stateMachine = new CrawlidStateMachine(this);
    }
    public void Direction()
    {
        stateMachine.Direction();
    }
    public void ChangeHealth()
    {
        stateMachine.ChangeHealth();
    }
    public void Update(GameTime _gameTime)
    {
        //TODO impliment actuall state changes
        frameCounter++;
        if (frameCounter >= 500)
        {
            state++;
            stateMachine.Update(_gameTime);
            if (state == 4)
            {
                state = 0;
            }
            frameCounter = 0;
        }
        Sprite.Update(_gameTime);
    }
```

Review

```
    public void Draw(SpriteBatch _spriteBatch, SpriteEffects _spriteEffects)
    {
        Sprite.Draw(_spriteBatch, _spriteEffects);
    }
}
```

```
public class CrawlidStateMachine
{
    private Crawlid CurrentCrawlid;
    public CrawlidStateMachine(Crawlid _enemy)
    {
        CurrentCrawlid = _enemy;
    }

    public void Direction()
    {
        CurrentCrawlid.left = !CurrentCrawlid.left;
    }

    public void ChangeHealth()
    {
        CurrentCrawlid.alive = !CurrentCrawlid.alive;
    }

    public void Update(GameTime _gameTime)
    {
        if (CurrentCrawlid.state == 1)
        {
            CurrentCrawlid.Sprite = SpriteFactory.Instance.CreateCrawlidTurnSprite(CurrentCrawlid.position);
        }

        if (CurrentCrawlid.state == 2)
        {
            CurrentCrawlid.Sprite = SpriteFactory.Instance.CreateCrawlidDeathAirSprite(CurrentCrawlid.position);
        }

        if (CurrentCrawlid.state == 3)
        {
            CurrentCrawlid.Sprite = SpriteFactory.Instance.CreateCrawlidDeathLandSprite(CurrentCrawlid.position);
        }

        if (CurrentCrawlid.state == 4)
        {
            CurrentCrawlid.Sprite = SpriteFactory.Instance.CreateCrawlidIdleSprite(CurrentCrawlid.position);
        }
    }
}
```

Readability Assessment

What IS readable:

The general structure of Crawlid.cs follows the generic pattern for an enemy, so any team member who understands how an enemy should behave can understand this class. The Update method reads logically for functionality sense for what needs to be accomplished for sprint 2. The use of SpriteEffects in the draw method will help in the future when trying to implement full enemy functionality.

In the second screenshot, CrawlidStateMachine.cs, the code is very readable and functions are short and simple, making it easy to interpret each function's purpose. Formatting and white space also make for smooth readability for debugging. This format is good for scalability as we make progress and incorporate more into the project.

What is NOT readable:

The naming inconsistency is the biggest readability issue. Most fields use the _camelCase convention, but "left" and "alive" drop the underscore. This forces the reader to do extra mental parsing, are these different from the other fields. Although update is proper for what sprint 2 is asking for, it is hard for an outside reader to understand how the crawlid is supposed to change. Finally, the class is missing XML documentation entirely

Crawlid states in the Update function have very similar code, this is an indication that there is probably a more scalable way to format the code. Professor Painter suggested using an enum or dictionary to easily store the states, further simplifying the file's contents.

Review Comments

Praise	**File:** `Crawlid.cs` (Draw method) Great use of SpriteEffects in the draw method. Setting this up will help in the future when enemy functionality is brought in. **File:** `CrawlidStateMachine.cs` (Overall structure) Great work making readable functions with understandable function names. White space also makes reading the code easy for debugging.
Must Fix	**File:** `Crawlid.cs` (Line 30 — boundary check) Missing the namespace at the top of the file. Missing XML documentation on what the class does. Add comments for future reference on what needs to be changed or will change. **Suggestion:** ***Add comments and XML documentation.***
Should Fix	**File:** `CrawlidEnemy.cs` (Lines 12–13) Fields like “left”, “alive”, and others are missing the underscore prefix. The class mixes two conventions, some fields use _prefix, some do not. This inconsistency hurts readability. **Suggestion:** *Rename to _left and _alive.* **File:** `CrawlidStateMachine.cs` (Update function) Currently, the update function has multiple if statements in a switch-case like format. The code within the conditionals are also similar, so there is likely a way to abstract this code to simplify the file’s contents as well as make the state selection more scalable for future reference. **Suggestion:** *Use enums or dictionaries to store various states.*
Suggestion	**File:** `Crawlid.cs` This class will need to be similar to a MovingAnimatedSprite (movement logic, animation cycling, bouncing). Consider inheriting from or composing with the existing class and adding only the enemy-specific behavior (alive, flip on direction change). This would reduce duplication and ensure bug fixes in the base logic propagate to all enemies.

4 Niko: Role as Project Manager

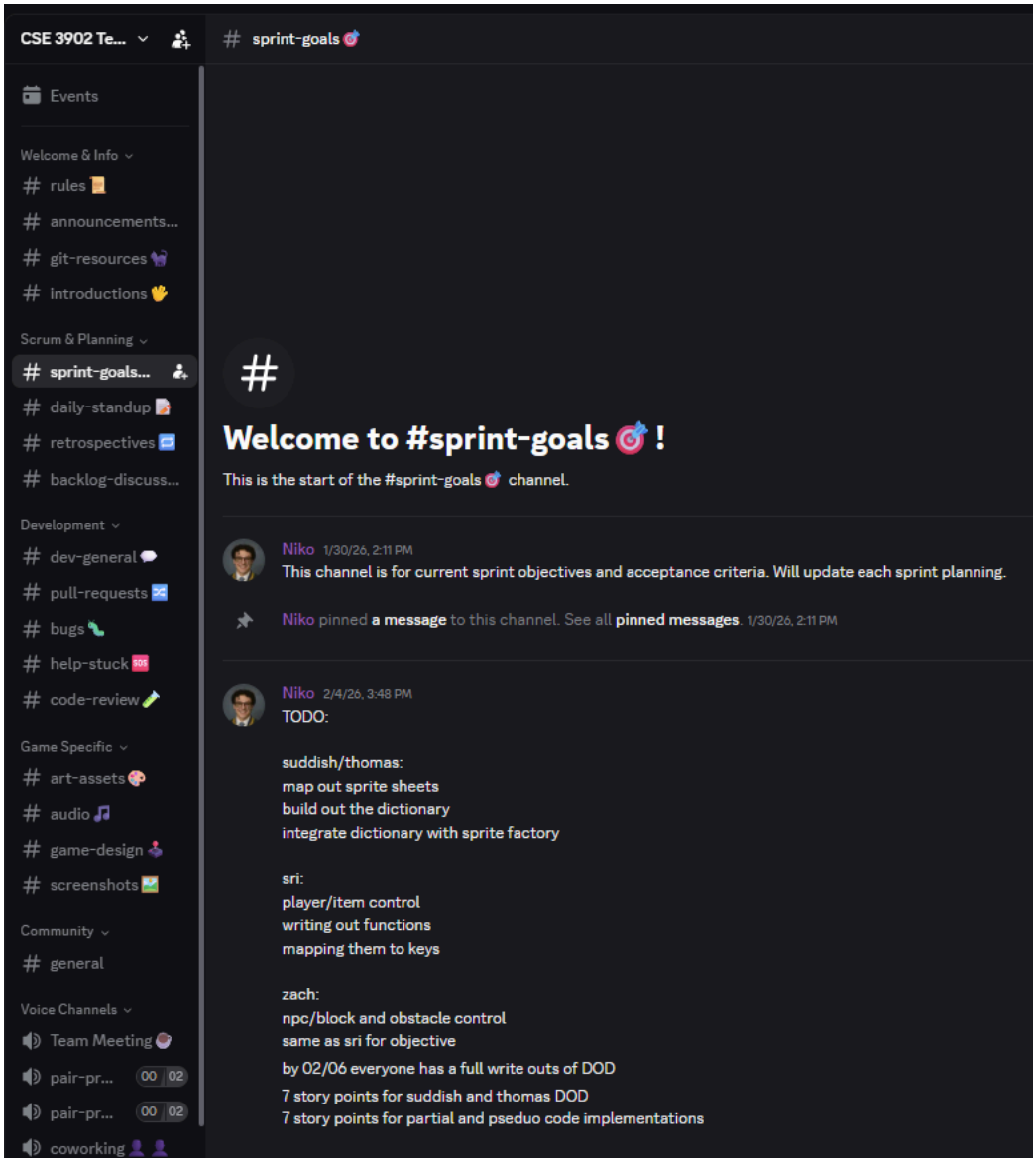
Field	Details

Author of "Code" Review	Thomas Sobodosh, Sudhish Gopalakrishnan
Date of "Code" Review	February 20, 2026
Sprint Number	Sprint 2
.cs File Reviewed	`N/A`
Author of .cs File	N/A
Time to Complete Review	40 minutes

Special note for this review: Niko's main role throughout this project so far has been as a project manager, therefore despite not having much code written for this Sprint, he has been a vital member in reaching what we've achieved. Niko has been primarily delegating tasks, reviewing progress throughout the sprint, and preparing structured formats for the group to follow - both communication wise and code wise.

Praise:

Niko's done a great job at organizing many aspects of the project by creating a user-friendly discord to have easy access to development resources, github resources, planning, and general information to increase accessibility/understanding in the group. Moreover, throughout the early stages of planning who's developing what in the sprint, Niko took responsibility in writing very useful pseudo-code to guide the team with the implementation. I can't seem to find an example of previously existing pseudo-code, otherwise I'd put it here, but it was very understandable and concise with examples of what some code may look like. Additionally, Niko has a lot of experience in git therefore he's a good resource to come to with questions. With much industry experience, Niko is organizing our approach as an agile project development. He does a great job of explaining the aspects of this form of project development to make sure the group understands its structure, advantages, and routines. Additionally, Niko has done a good job of distributing work and always coming up with things that can be done to ensure the team members know exactly what to be working on in terms of development. The Discord screenshot below demonstrates the layout of the team's Discord where Niko laid out the frameworks of how the community is separated up into specific subsections to ask questions, chat with others, brainstorm ideas, and share information.



Areas for improvement:

Generally, I wish there was more structure with when the group meets, either in person or online, as I feel this will help keep our group focused - therefore maybe directing a discussion on what time works would be useful. This goes in hand with that, but to have Niko check up and ask how things are going more directly/formally- this could be during those meetings, not to force work, but just make sure progress is being made to hold the team members accountable. I feel like these both fit into an area that Niko can manage/be aware of as the project manager. Finally, the only other area for improvement could be better division of labor for code reviews. Just specifically, coming up with a plan for who will review who and when generally sounds like something that could be useful for the team.